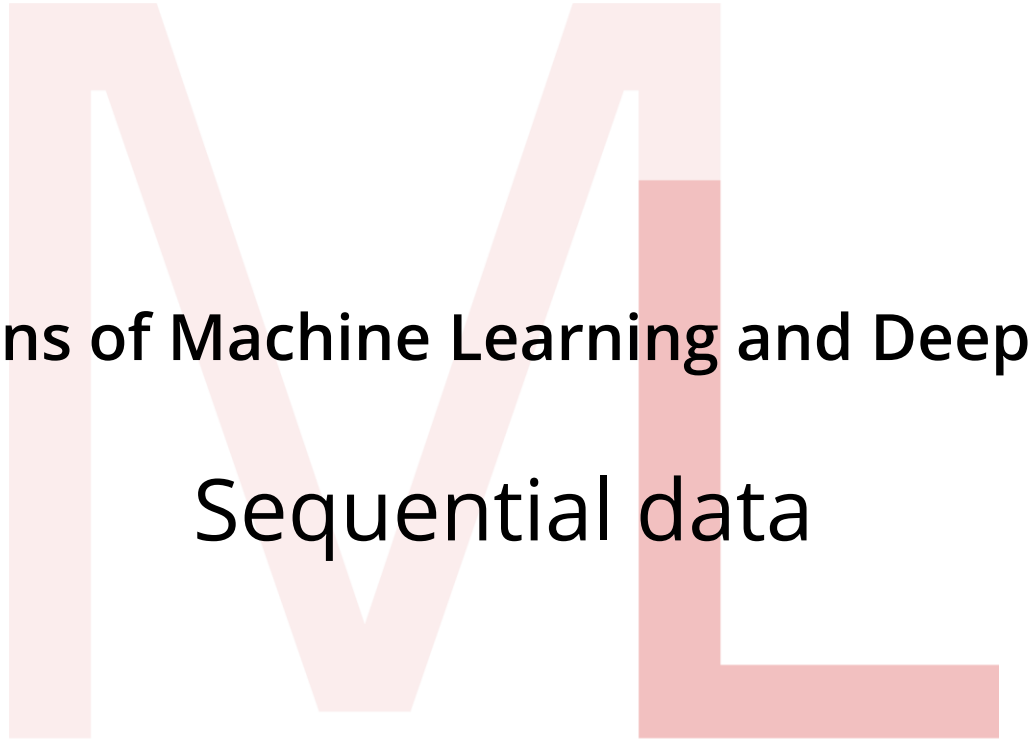
Foundations of Machine Learning and Deep Learning

Sequential data



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Sequential data

- Working with sequences
  - Inputs are ordered lists of feature vectors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T$
  - each feature $\boldsymbol{x}_t$ indexed by a time step $t \in \mathbb{Z}^+$ lying in $\mathbb{R}^d$

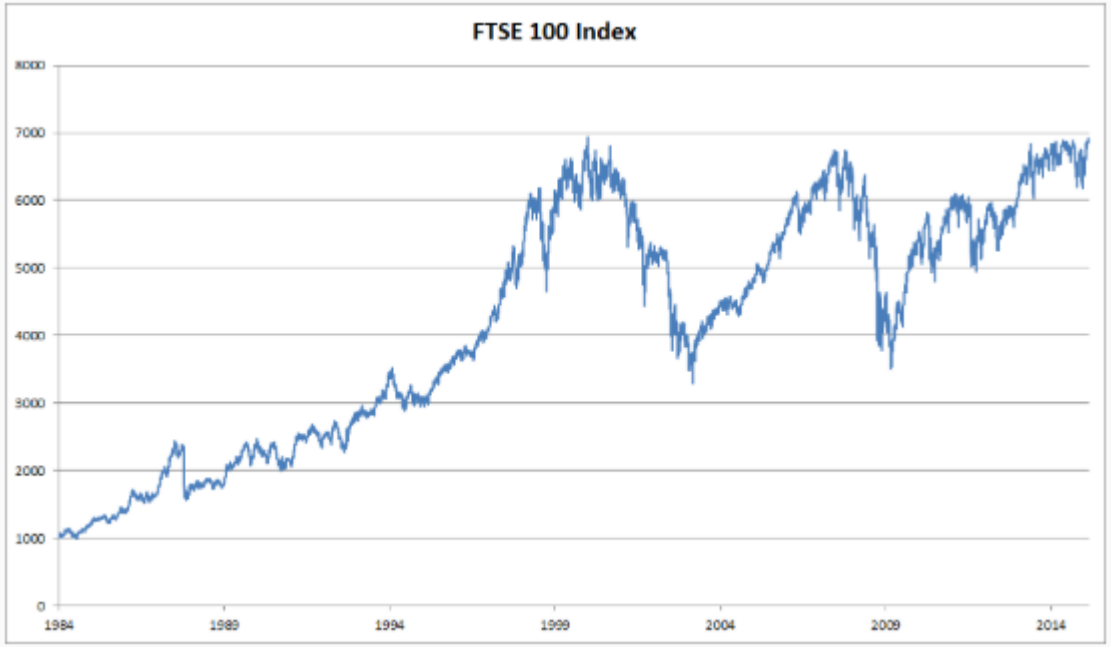
- Distributions
  - Individual inputs were assumed to be sampled independently from the same distribution $P(X)$
  - Complete sequences are sampled independently, but data at each time step are dependent of each other

Sequential data

- Sequential data refers to data points that are ordered and where the order matters.
- **Examples**:
  - Time series: stock prices, temperature measurements
  - Text: letters or words in a sentence
  - Audio: waveform signals
  - Video: sequence of frames
- The meaning of a sequence often depends on the order of its elements. For instance:
  - “dog bites man” vs. “man bites dog” contain the same words but in a different order, leading to very different meanings
  - In a time series, predicting tomorrow’s temperature requires knowledge of past days, not just a random set of values

Autoregressive models

- Models that regress the value of a signal on the previous values of that same signal
- **Example**
  - Predict the stock prize data from the FTSE 100 index without extra information but the previous values



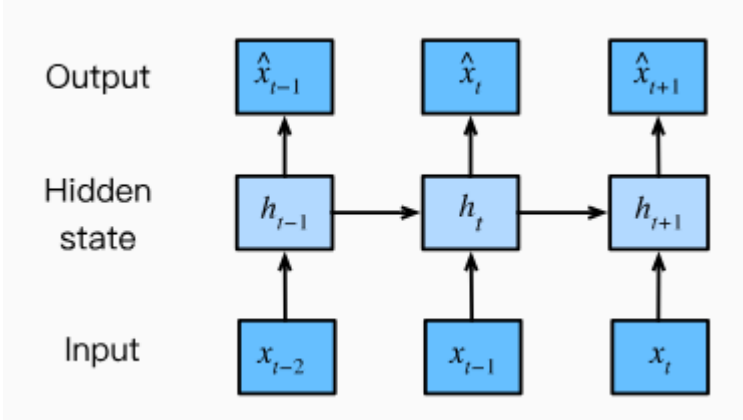
Autoregressive models

- Input size?
  - The number of inputs $x_{t-1}, \dots, x_1$ varies depending on t
- Possible solutions
  - Condition on some window of length τ and only use $x_{t-1}, \dots, x_{t-\tau}$ observations

$$\hat{x} = P(x_t | x_{t-1}, \dots, x_{t-\tau})$$

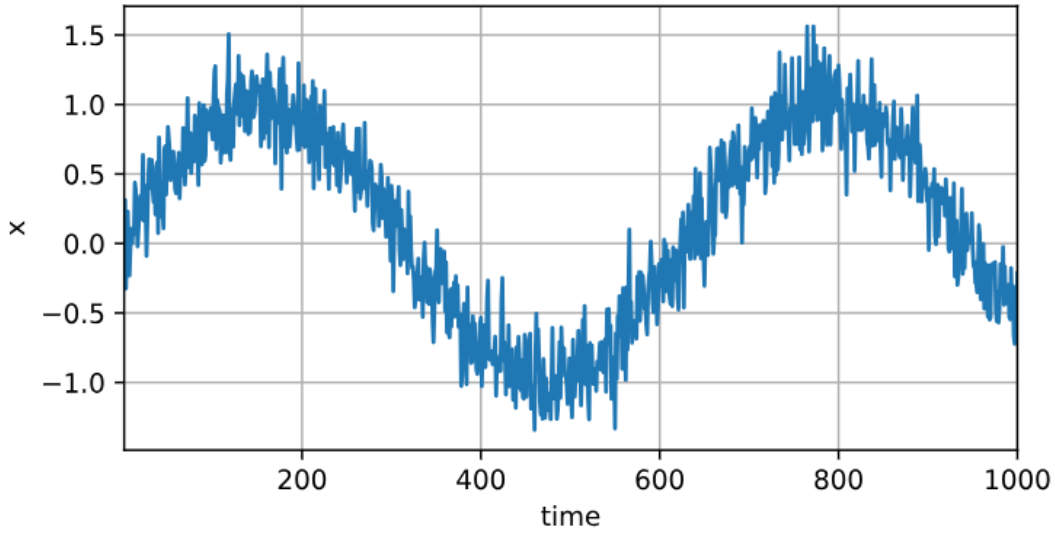
  - Maintain some summary h_t of the past observations and update it at every step. h_t is never observed → latent autoregressive model

$$\hat{x} = P(x_t | h_t)$$
$$h_t = g(h_{t-1}, x_{t-1})$$

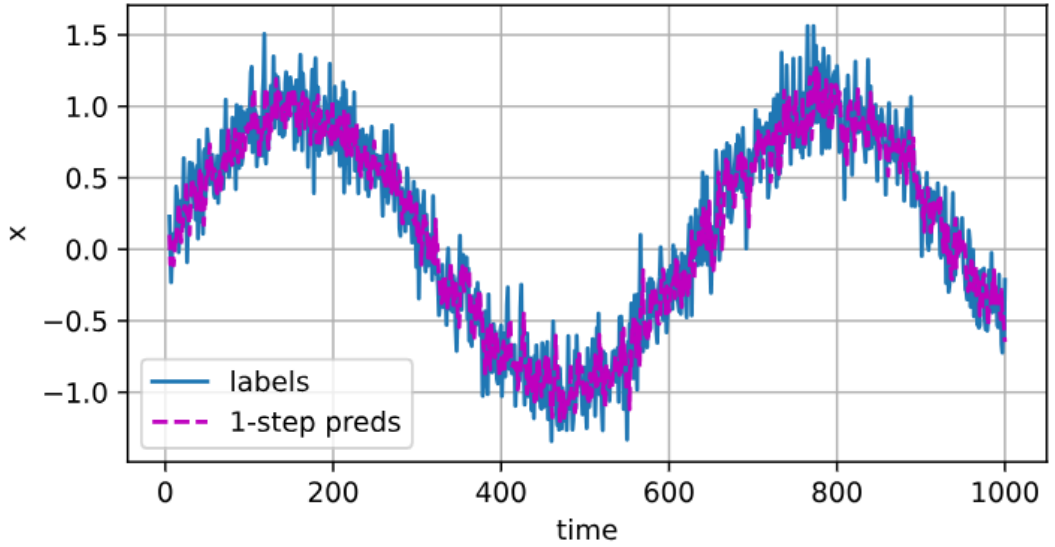


Autoregressive models

- **Example**
  - Synthetic data following sin function corrupted with additive noise

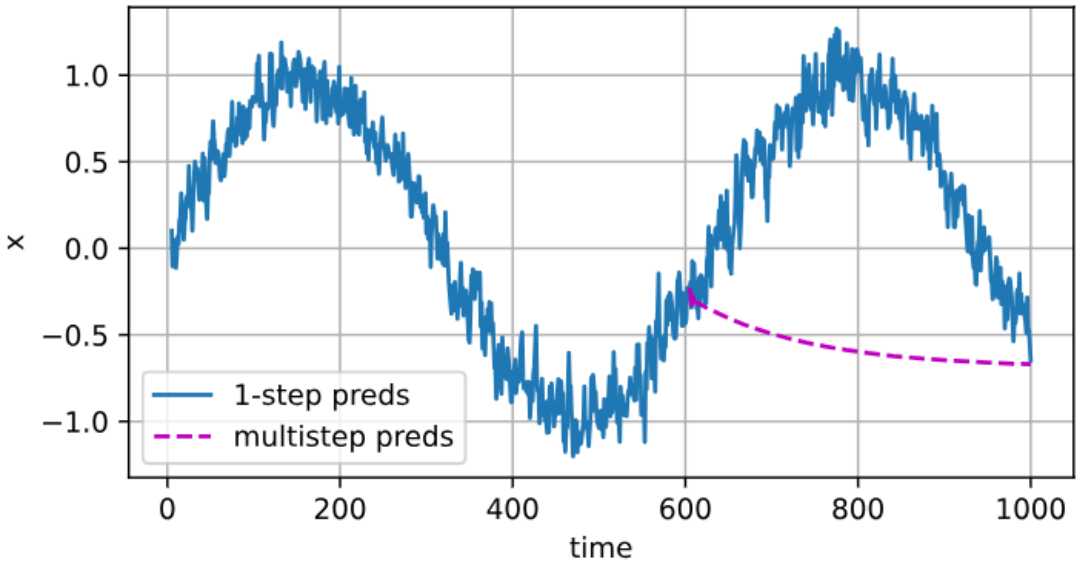


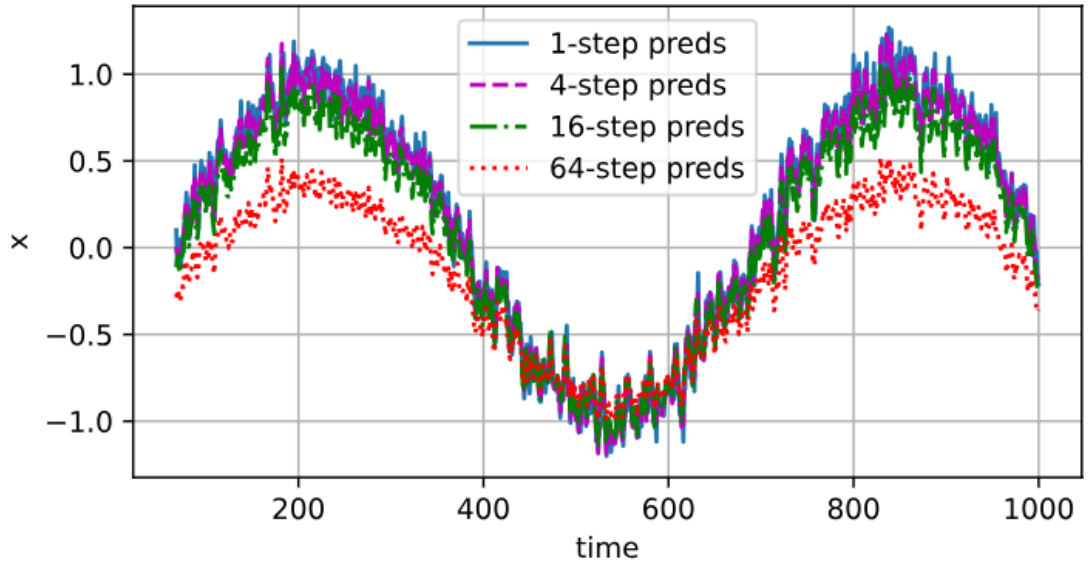
  - Prediction of the next value at each time step



Autoregressive models

- **Example**
  - Prediction at each time step based on previous predictions
  - Prediction at each time step based on previous (more previous) data





Working with text

- Usual working Flow
  1. Load text as strings into memory
  2. Split the strings into tokens (words or characters)
  3. Build a vocabulary dictionary to associate each vocabulary element with a numerical index
  4. Convert the text into sequences of numerical indices

Working with text

2 - Split the strings into tokens (words or characters)

- Tokens are atomic units of text
- Each time step corresponds to 1 token
- What constitutes a token is a design choice
  - Characters
  - Words

Working with text

3 – Build a vocabulary dictionary

- The inputs to the model must be numerical
- Associate each distinct token value with a unique index
    1. Based on the training corpus determine the set of unique tokens
    2. Assign a numerical index to each unique token
    3. Rare tokens are often dropped
        - Represented by special <unk> token (unknown value)

Working with text

- **Example** (character tokenization)
  - text = “hello”
  - Vocabulary = {“h”, “e”, “l”, “o”}
  - Assign index to characters = {“h”→0; “e”→1; “l”→2; “o”→3}
  - Codification: “hello”→ [0, 1, 2, 2, 3]

Embeddings

- Encoding of each token:
  - An integer number
  - One-hot encoding
  - Embeddings

- Order relation does not make sense
- Embeddings are more compact than one-hot vectors
- Embeddings can contain meaning
  - Pretrained embeddings available
  - Trained (or not) with the training data

Language models

- The goal of language models is to estimate the joint probability of a whole text sequence of length T

$$P(x_1, x_2, \dots, x_T)$$

- Generate natural text on its own
- Even more: generate a meaningful dialog conditioning the text on previous dialog fragments
- More limited language models
  - Reduce ambiguity in speech recognition ("to recognize speech" vs "to wreck a nice beach"
  - Help in summarization algorithms ("I want to eat grandma" vs "I want to eat, grandma")

Perplexity

- Measure the quality of a language model
- **Example**: three models continue the sentence “It is raining”
  - Model 1: “It is raining outside”
  - Model 2: “It is raining banana tree”
  - Model 3: “It is raining piouw;kcj pwepoiut”
- Measure how surprising the text is

Perplexity

- Computing the likelihood of the sequence

$$L = \prod_{t=1}^{n} P(x_t | x_1, \dots, x_{t-1})$$

- If the sequence is long, this number is too small → logarithm

$$logL = \sum_{t=1}^{n} \log P(x_t | x_1, \dots, x_{t-1})$$

- For the measure to be comparable in texts of different length → average

$$Avg \log L = \frac{1}{n} \sum_{t=1}^{n} \log P(x_t | x_1, \dots, x_{t-1})$$

Perplexity

- Change of scale → interpretation: number of effective options

$$PPL = \exp(-Avg \log L) = -\frac{1}{n}\sum_{t=1}^{n} \log P(x_t | x_1, \ldots, x_{t-1})$$

- Best case scenario: $PPL = 1$ (the model always estimates the probability of the target token as 1)
- Worst case scenario: $PPL = +\infty$ (the model always estimates the probability of the target token as 0)
- Baseline: $PPL = \#unique\ tokens$ (the model predicts a uniform distribution over all available tokens in the dictionary)